

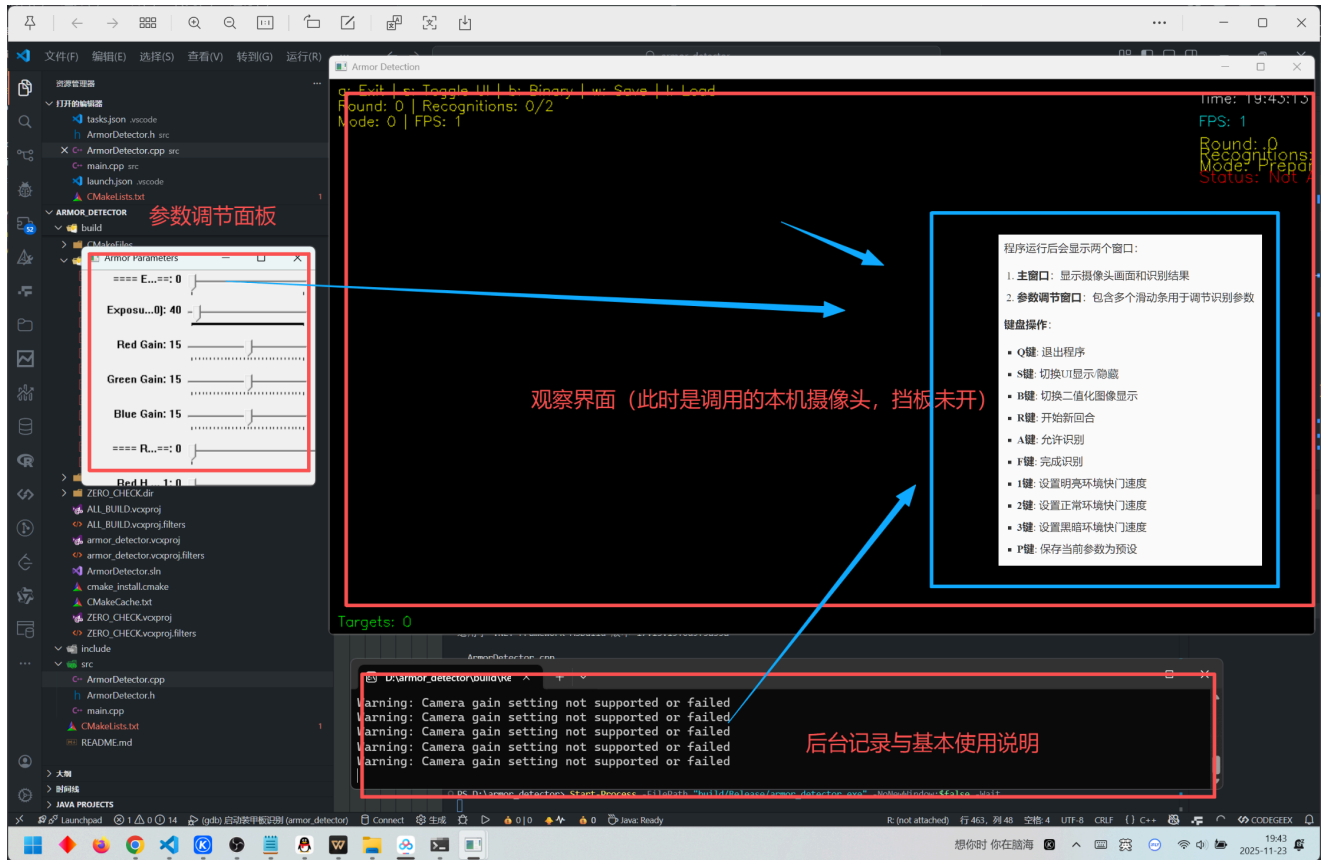
RoboMaster Armor Detector

目录

- RoboMaster Armor Detector
 - 目录
- RoboMaster装甲板识别系统
 - 功能特性
 - 系统要求
 - 构建说明
 - Windows环境
 - Linux/Ubuntu环境
 - 使用说明
 - 运行程序
 - 操作说明
 - 参数调节
 - 图像处理参数
 - 相机参数
 - 游戏控制参数
 - 比赛流程
 - 准备阶段
 - 正式比赛
 - 系统架构
 - 核心组件
 - 处理流程
 - 技术细节
 - 图像处理原理
 - 灯条特征筛选
 - 装甲板配对
 - 性能基准
 - 性能优化
 - 故障排除
 - 常见问题及解决方案
 - 贡献指南
 - 许可证

RoboMaster装甲板识别系统

本项目是一个基于C++和OpenCV4开发的RoboMaster比赛用装甲板识别系统，用于机器人视觉自动瞄准，提升对抗中的精准打击能力。系统利用装甲板灯条发光的特性，通过低曝光设置屏蔽环境光干扰，采用二值化处理获取灯条图像，并基于灯条的几何特征进行筛选和匹配，最终实现装甲板的精确识别。



功能特性

1. **实时装甲板识别**：支持红色和蓝色双色装甲板的实时识别
2. **精确位置计算**：计算并输出装甲板中心坐标，供后续控制系统使用
3. **可视化UI界面**：提供可视化界面，实时显示图像处理结果
4. **参数调节面板**：内置参数调节面板，支持曝光、阈值等参数动态调整
5. **多种摄像头支持**：支持多种摄像头输入源（如USB摄像头、网络流等）
6. **回合制识别控制**：支持比赛规则的回合制识别控制
7. **相机参数调节**：支持调节相机参数：曝光时间(MVExpTime)、RGB增益(r_gain, g_gain, b_gain)
8. **快门速度预设**：支持快门速度预设功能，适应不同光照条件
9. **DF100 USB相机优化**：专门针对DF100 USB相机（1280×720分辨率）进行优化
10. **分辨率自适应**：UI显示自动适配不同分辨率，确保良好可视效果

系统要求

- C++11 或更高版本

- OpenCV 4.0 或更高版本
- CMake 3.10 或更高版本
- 支持C++11的编译器（如GCC、Clang、MSVC等）

构建说明

*Windows*环境

1. 安装Visual Studio Build Tools并选择"C++ build tools"工作负载，或安装Visual Studio/MinGW-w64作为编译器
2. 安装CMake并将安装路径添加到系统PATH
3. 下载并解压OpenCV预编译库到指定目录
4. 将OpenCV的bin目录（如[OpenCV根目录]/x64/[编译器]/bin）添加到系统PATH，确保运行时能加载所需dll
5. 使用CMake配置OpenCV时，必须设置OpenCV_DIR环境变量或CMake变量，指向包含OpenCVConfig.cmake文件的目录

构建命令：

```
mkdir build
cd build
cmake ..
cmake --build . --config Release
```

*Linux/Ubuntu*环境

安装依赖：

```
sudo apt-get update
sudo apt-get install build-essential cmake git
sudo apt-get install libopencv-dev
```

构建项目：

```
mkdir build
cd build
cmake -DOpenCV_DIR="path/to/opencv" ..
make # Linux
# 或
cmake --build . --config Release # Windows
```

使用说明

运行程序

```
./armor_detector          # 使用默认摄像头(0)
./armor_detector 1        # 使用摄像头(1)
```

.\armor_detector.exe

操作说明

程序运行后会显示两个窗口：

1. **主窗口**：显示摄像头画面和识别结果
2. **参数调节窗口**：包含多个滑动条用于调节识别参数

键盘操作：

- **Q键**: 退出程序
- **S键**: 切换UI显示/隐藏
- **B键**: 切换二值化图像显示
- **R键**: 开始新回合
- **A键**: 允许识别
- **F键**: 完成识别
- **1键**: 设置明亮环境快门速度
- **2键**: 设置正常环境快门速度
- **3键**: 设置黑暗环境快门速度
- **P键**: 保存当前参数为预设

参数调节

参数调节窗口包含以下滑动条：

图像处理参数

- **HSV颜色阈值**：调节红蓝装甲板识别的HSV范围
 - 红色阈值分为两组（由于HSV色彩空间中红色横跨0°和180°）
 - 蓝色阈值单独一组
 - 每组包含H(色调)、S(饱和度)、V(明度)的高低值
- **形状过滤参数**：
 - **Min Area**: 最小轮廓面积（像素）
 - **Max Area**: 最大轮廓面积（像素）

- **装甲板匹配参数:**
 - **Min Distance:** 灯条间最小距离
 - **Max Distance:** 灯条间最大距离
 - **Max Angle Diff:** 灯条间最大角度差

相机参数

- **Exposure Time (x100):** 调节相机曝光时间（快门速度），范围1-100000微秒，实际值为显示值×100
- **Red Gain, Green Gain, Blue Gain:** 调节相机RGB通道增益，范围0-30
- **Preset Bright/Normal/Dark:** 预设的快门速度参数，实际值为显示值×100

游戏控制参数

- **Recognition Mode:** 切换识别模式（准备阶段、第一次识别、第二次识别）

比赛流程

准备阶段

1. 调节参数适应环境光照
2. 设置预设参数
3. 测试识别效果

正式比赛

每回合最多执行两次识别任务：

1. **第一次识别:** 回合开始时装甲板点亮
2. **第二次识别:** 兑换区申请，成功识别可获得额外增益

系统架构

核心组件

- **ArmorDetector类:** 负责图像处理流程，包括颜色空间转换、二值化、轮廓分析、匹配判断等
- **OpenCV库:** 提供图像读取、显示、滤波、形态学操作等功能
- **UI模块:** 通过OpenCV内置GUI实现参数实时调节

处理流程

- 相机读图(逐帧读取视频流)
- 图像预处理(HSV颜色空间转换、红/蓝色阈值分割、形态学滤波去噪)
- 特征提取(轮廓检测找到候选区域、灯条特征筛选、装甲板配对)
- 输出结果(绘制装甲板中心点和边界框、绘制颜色和坐标文本、终端输出结果)

技术细节

图像处理原理

系统采用计算机视觉技术进行装甲板识别，主要流程包括：

- 颜色空间转换**：将BGR图像转换为HSV色彩空间，便于颜色分离
- 颜色阈值分割**：根据HSV值分离出红蓝装甲板区域
- 形态学操作**：使用开运算和闭运算去除噪声并填充空洞
- 轮廓检测**：寻找图像中的连通区域边界
- 特征筛选**：根据灯条的几何特征筛选候选区域
- 装甲板匹配**：将符合条件的灯条配对形成装甲板

灯条特征筛选

```
bool isLightBar(const vector<Point>& contour) {  
    double area = contourArea(contour);  
    if (area < min_area || area > max_area) return false; // 面积筛选  
    RotatedRect rect = minAreaRect(contour);  
    float width = rect.size.width;  
    float height = rect.size.height;  
    if (width < height) swap(width, height);  
    float aspect_ratio = width / height;  
    if (aspect_ratio < min_ratio || aspect_ratio > max_ratio) return false; // 长宽比筛选  
    return true;  
}
```

筛选标准：

- 面积范围：默认80-8000像素（可调节）
- 长宽比：默认1.5-8之间，确保为细长形状（可调节）

装甲板配对

```
vector<Point2f> pairLightBars(const vector<RotatedRect>& light_bars) {
    vector<Point2f> armor_centers;
    for (size_t i = 0; i < light_bars.size(); i++) {
        for (size_t j = i + 1; j < light_bars.size(); j++) {
            Point2f center1 = light_bars[i].center;
            Point2f center2 = light_bars[j].center;
            RotatedRect rect1 = light_bars[i];
            RotatedRect rect2 = light_bars[j];

            // 计算灯条高度
            float height1 = max(rect1.size.width, rect1.size.height);
            float height2 = max(rect2.size.width, rect2.size.height);
            float min_height = min(height1, height2);

            // 计算两个灯条之间的距离
            float distance = norm(center1 - center2);

            // 距离应该与灯条高度成比例
            if (distance < 2 * min_height || distance > 8 * min_height) {
                continue;
            }

            // 计算角度差（考虑OpenCV的角度特性）
            float angle1 = rect1.angle;
            float angle2 = rect2.angle;
            if (angle1 < -45) angle1 += 90;
            if (angle2 < -45) angle2 += 90;
            float angle_diff = abs(angle1 - angle2);
            if (angle_diff > 15) {
                continue;
            }

            // 计算水平和垂直距离
            float horizontal_dist = abs(center1.x - center2.x);
            float vertical_dist = abs(center1.y - center2.y);
            // 水平距离应大于垂直距离（装甲板通常水平排列）
            if (horizontal_dist < vertical_dist) {
                continue;
            }

            // 计算装甲板中心
            Point2f armor_center((center1.x + center2.x) / 2,
                                  (center1.y + center2.y) / 2);
            armor_centers.push_back(armor_center);
        }
    }
    return armor_centers;
}
```

配对条件:

- 灯条间距: 与灯条高度成比例 (2-8倍最小灯条高度)
- 灯条角度差: <15度
- 水平距离大于垂直距离

性能基准

在以下测试环境下进行了性能评估：

- 硬件平台：Intel i7-8700K CPU @ 3.70GHz, 16GB RAM
- 摄像头：DF100 USB相机 (1280×720@60fps)
- 操作系统：Windows 10 64-bit
- 编译器：MSVC 2019

场景	平均处理时间	FPS
简单背景	8ms	125
复杂背景	15ms	67
多目标识别	25ms	40

性能优化

- 在轮廓筛选阶段增加更多约束条件以减少候选灯条数量
- 装甲板匹配可引入空间索引或KD-Tree加速搜索
- 红蓝颜色识别可并行处理提升效率
- 重用图像缓冲区减少内存分配开销

故障排除

常见问题及解决方案

1. 无法打开摄像头

- 检查摄像头是否正确连接
- 确认设备管理器中是否有摄像头驱动
- 尝试更换USB端口
- 检查是否有其他程序正在使用摄像头

2. 识别率低

- 调整曝光时间至合适值（通常较低）
- 调整HSV颜色阈值以适应当前光照条件
- 确保装甲板与摄像头距离适中（建议1-3米）
- 检查灯条特征筛选参数是否合适

3. 程序崩溃

- 检查OpenCV库是否正确链接
- 确认输入图像尺寸符合预期
- 查看控制台输出的错误信息

- 确保有足够的系统资源（内存、CPU）

4. 参数调节无响应

- 确保参数调节窗口处于激活状态
- 检查是否有其他程序占用了键盘输入
- 重启程序

5. 图像显示异常

- 检查摄像头分辨率设置是否正确
- 确认摄像头驱动是否正常工作
- 尝试降低图像处理参数以提高性能

调试技巧

1. 使用二值化图像显示：

- 按B键切换二值化图像显示
- 观察红蓝装甲板是否被正确二值化
- 根据二值化效果调整HSV阈值参数

2. 参数调整顺序：

- 首先调整曝光时间，确保图像不过曝或欠曝
- 然后调整HSV阈值，使装甲板灯条清晰可见
- 最后调整形状过滤和匹配参数

3. 保存参数配置：

- 调试完成后按P键保存当前参数
- 参数将保存到params.xml文件中
- 下次启动时会自动加载保存的参数

![image-20251123214335636]

(C:\Users\yohoten\AppData\Roaming\Typora\typora-user-images\image-20251123214335636.png)

